

6. COMPLEXIDADE DO PROBLEMA

- Até aqui, neste trabalho, o centro das atenções foi o algoritmo. Neste capítulo, o ponto de referência deixará de ser o algoritmo para ser o **problema**.
- O objetivo é estudar as limitações dos problemas com respeito à complexidade dos algoritmos que os resolvem.
- O que se deve pensar antes de codificar um problema?
 - o O problema é computável?
 - o Será que existe solução (algoritmo) eficiente para este problema?
- A questão da computabilidade foi proposta por Hilbert em 1900, que conjecturava que todo problema bem posto seria demonstrável.
- Em 1931, Gödel apresentou o “**Teorema da Completude**”, mas ainda havia a questão formal da definição de computabilidade, que foi fechada em 1936, com a apresentação e a aceitação da “**Tese de Church**”.

- A questão $P \times NP$ surgiu em 1971, com o artigo “The complexity of theorem-proving procedures”, e tem sido intensamente estudada. A conjectura que existe é, “ $P \neq NP$ ”. A partir dessa conjectura, foi criada toda a teoria, na qual a figura principal é a classe NP-Completa.

Visão geral do capítulo

- 6.1 Limite de complexidade de um problema.

Apresenta os conceitos referentes à complexidade, como limite superior de complexidade de um problema e complexidade mínima de problema.

- 6.2 Intratabilidade.

Define, relaciona e ilustra com exemplos as principais classes de problemas, como : P , NP , NP -Completo, $Co-NP$ e NP -Difícil.

6.1 LIMITE DE COMPLEXIDADE DE UM PROBLEMA

- O **limite superior** de complexidade de um problema refere-se ao **melhor algoritmo que o resolve**. Por exemplo, para o problema de Classificação, no capítulo 3, apresenta-se o algoritmo 3.3.3: Classificação de Vetor, com complexidade $O(n^2)$. Já no capítulo 5, o algoritmo do exemplo 5.2.3: *Mergsort* tem complexidade $O(n \log_2 n)$, e $n \log_2 n$ é um limite superior para a Classificação, no sentido que é possível classificar qualquer lista com uma complexidade $O(n \log_2 n)$.
- O **limite inferior** de um problema refere-se à **melhor complexidade possível**. Os problemas têm suas dificuldades, que não permitem o desenvolvimento de algoritmos melhores do que um certo limite de complexidade. Um limite inferior **é um resultado teórico** que determina não ser possível desenvolver um algoritmo para o problema com complexidade melhor que uma certa função. Esta função então é um *limite inferior de complexidade do problema*.
- A técnica mais simples de **cálculo do limite inferior** é simplesmente **contar as entradas e as saídas produzidas**, pois o algoritmo precisa, no mínimo, ler a entrada e dar as saídas.

- Um limite inferior pode ser obtido sem, entretanto, conhecer-se um algoritmo que realmente atinja essa complexidade. Nesse caso, o limite inferior conhecido não é igual ao limite superior conhecido.
- Enquanto houver diferença entre os limites de complexidade inferior e superior para o problema, a questão pode progredir nos dois sentidos:
 - o Um algoritmo melhor que o limite superior pode ser desenvolvido, ou um cálculo mais apertado da complexidade mínima pode ser alcançada, diminuindo a diferença entre esses dois limites.
 - o Quando essa **diferença desaparece**, a complexidade mínima do problema é conhecida. Então, a complexidade mínima do problema é a melhor complexidade que um algoritmo que resolve esse problema pode atingir. O problema do cálculo da complexidade, nesse caso, é dito **fechado**.

Problema de Classificação

- O **problema de classificação** por comparação é um problema com limite de complexidade determinada, **$n \log n$** .
- Não existe algoritmo de classificação baseado em comparações de complexidade melhor que $n \log n$.
- Consideremos três elementos a, b, c , que desejamos classificar.
 - o A única informação disponível a respeito é uma relação de ordem ($<$) entre os elementos.
 - o Nesse caso, os únicos algoritmos possíveis são baseados em comparação.
 - o A árvore a seguir é um exemplo de árvore de decisão para o problema.

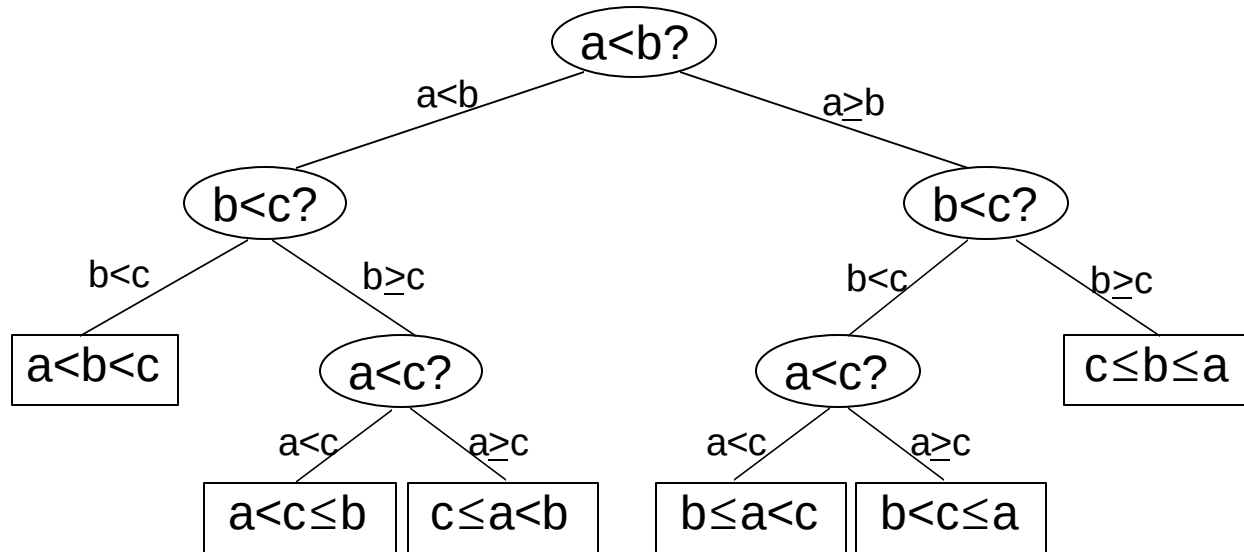


Figura 6.1.1: Árvore de comparações

- Qualquer **algoritmo de classificação** baseado em comparação pode ser representado por uma **árvore de decisão**, cujas folhas são as possíveis classificações dos elementos, isto é, todas as $n!$ permutações possíveis, onde n é o número de elementos.
- O número de comparações necessárias para classificar n elementos é a profundidade, na árvore de decisão, da folha que ilustra o caso. A complexidade no pior caso é a maior profundidade de folha na referida árvore.

Complexidade no Pior Caso

- Árvore binária completa possui relação: folhas = 2^{altura}
- No problema em questão:
 - o **folhas**: são $n!$ permutações possíveis;
 - o **altura**: no pior caso, $c(n)$
- Neste caso a relação passa a ser:

$$n! \leq 2^{c(n)}$$

$$\log_2 n! \leq \log_2 2^{c(n)}$$

$$c(n) \geq \lceil \log_2 n! \rceil$$

- Usando, a aproximação de Stirling

$$[\log_2 n!] = n \log_2 n - \frac{n}{(\ln 2)} + \frac{1}{2} \log_2 n + O(1)$$

como, $\ln 2 = \log_e 2$ é constante e $n \log_2 n$ absorve os outros termos, então:

$[\log_2 n!]$ é $O(n \log_2 n)$, e $c(n)$ é $O(n \log_2 n)$

- Para qualquer **problema de classificação** onde a única informação dos elementos a serem classificados é a **relação de ordem** entre eles, o **limite de complexidade no pior caso** é $n \log_2 n$, ou seja, **$n \log n$** .

Complexidade Média

- No caso médio com distribuição uniforme, mostra-se um resultado análogo.
- Suponha que as $n!$ entradas têm a mesma probabilidade: $\frac{1}{n!}$.
- A cada uma das $n!$ permutações corresponde uma folha na árvore de decisão, e qualquer um dos caminhos da raiz a uma folha é igualmente provável de ser percorrido.
- O número médio de comparações é a profundidade média das $n!$ folhas.
- Seja $S(A, k)$ a soma das profundidades de todas as folhas da árvore de decisão A , com k folhas.
- Mínimo da soma das profundidades: $m(k) = \min_A \{ S(A, k) \}$
- Profundidade média: $\frac{m(k)}{k}$

- Prova-se, por indução no número de nodos da árvore, que esse limite é $\log k$, ou $m(k) \geq k \log k$.

o **BI**: Para um nodo, a árvore é um nodo isolado, e $m(k) = 0$.

Por outro lado, $\log k = \log 1 = 0$, e $k \log k = 0$

o **HI**: Suponha $m(k') \geq k' \log k'$ para todo $k < k'$.

o **PI**: Seja uma árvore de decisão A . Sejam A_1 e A_2 as duas subárvores de A , imediatamente abaixo da raiz, com, respectivamente, k_1 e k_2 folhas, tal que $k_1 + k_2 = k$. A_1 e A_2 são árvores de decisão com menos nodos, portanto vale a hipótese de indução. A profundidade de um nodo qualquer nas subárvores A_1 e A_2 é uma unidade menor que a profundidade desse mesmo nodo em A , i.e.,

$$S(A, k) = k_1 + S(A_1, k_1) + k_2 + S(A_2, k_2), \text{ e}$$

$$m(k) = \min_{k_1+k_2=k} \{k + m(k_1) + m(k_2)\}$$

$$= k + \min_{k_1+k_2=k} \{m(k_1) + m(k_2)\}$$

o **Hipótese de Indução**, $m(k_1) \geq k_1 \log k_1$ e $m(k_2) \geq k_2 \log k_2$, então

$$m(k) \leq k + \min_{k_1+k_2=k} \{ k_1 \log k_1 + k_2 \log k_2 \}$$

o Para resolver

$$\min_{k_1+k_2=k} \{ k_1 \log k_1 + k_2 \log k_2 \}$$

substitua k_1 por x e k_2 por $k-x$, e defina

$$f(x) = x \log x + (k-x) \cdot \log (k-x).$$

o Derivando tem-se:

$$f'(x) = \log x + \frac{x}{x \ln 2} + (-1) \log(k-x) + (k-x) \frac{-1}{(k-x) \ln 2}$$

$$= \log x + \frac{1}{\ln 2} - \log(k-x) - \frac{1}{\ln 2} \quad (\ln 2 = \log_e 2)$$

$$= \log x - \log(k-x)$$

$$= \log\left(\frac{x}{k-x}\right), \quad \text{mas}$$

$$\log\left(\frac{x}{k-x}\right)=0 \Leftrightarrow \frac{x}{k-x}=1 \Rightarrow x = k-x \Rightarrow 2x = k \Rightarrow x = \frac{k}{2} \quad (\text{pois, } x \geq 0)$$

o Derivando novamente,

$$\begin{aligned} f''(x) &= \frac{1}{\left(\frac{x \cdot \ln 2}{k-x}\right)} \left(\frac{(k-x) - (-1)x}{(k-x)^2} \right) \\ &= \frac{k}{x(k-x)\ln 2} \end{aligned}$$

$$f''\left(\frac{k}{2}\right) = \frac{2}{k \ln 2} > 0 \quad \text{para } \forall x \in (0, k), \text{ então a função tem mínimo no ponto } \frac{k}{2}.$$

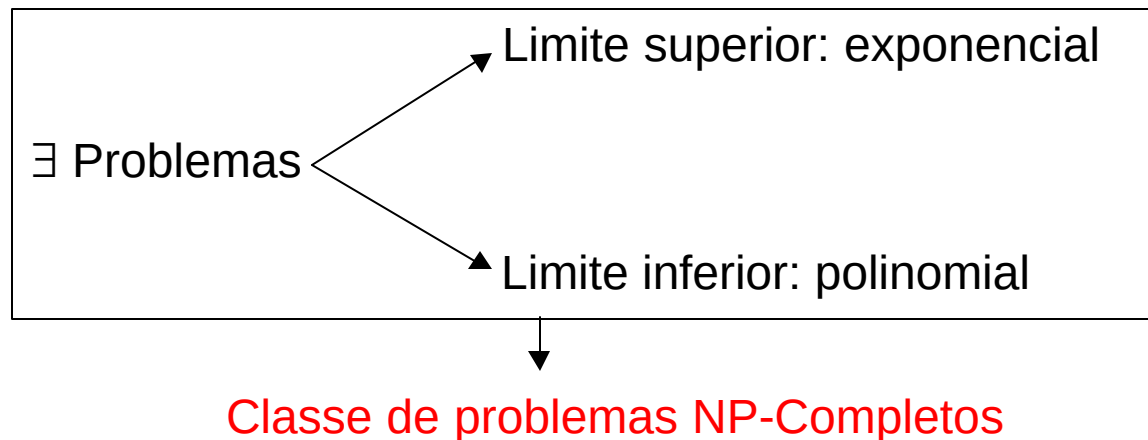
o Conclui-se que:

$$\begin{aligned}m(k) &\geq k + \frac{k}{2} \cdot \log \frac{k}{2} + \frac{k}{2} \cdot \log \frac{k}{2} \\&\geq k + k \log \frac{k}{2} \\&\geq k (\log k - \log 2 + 1) \\&\geq k \log k\end{aligned}$$

- Como a árvore tem $n!$ folhas, então $m(n!) \geq n! \cdot \log n!$, e a profundidade média mínima é $\frac{m(n!)}{n!} \geq \log n!$, que é, pela aproximação de Stirling, $O(n \log n)$.

6.2 INTRATABILIDADE

- Problema tratável: existe algoritmo eficiente, ou seja, **limite superior é polinomial**.
- Problema intratável: não tem algoritmo eficiente, ou seja, **limite inferior é exponencial**.
- Problemas que não satisfazem nenhuma das duas condições:



- Problemas NP-completos:
 - o Não se sabe se existe ou não algoritmo polinomial que o resolva;

- o Achar um algoritmo polinomial que resolva um problema NP-completo implica que todos os outros também possuem um algoritmo polinomial;
- o Acredita-se que não existe um algoritmo polinomial para os NP-completos.

6.2.1 Conceitos Básicos

⇒ **O que precisamos saber de um problema?**

- (i) Entradas (parâmetros)
- (ii) Quais propriedades a resposta ou solução deve satisfazer?
(Propriedades da solução)

⇒ **Tipos de problema:**

o Problema de decisão

- Se a pergunta tem **resposta SIM/NÃO**.
- Problema 6.2.1: *Problema da existência de Cobertura de Vértices*
 - o **Instância**: Um grafo $G=(V,E)$ e um natural k .
 - o **Pergunta**: **Existe V'** (subconjunto de V) tal que $|V'| \leq k$ e toda aresta de G tenha pelo menos uma extremidade em V' ?

o Problema de localização

- Exige a exibição de uma solução, envolvendo a procura de **um elemento que satisfaça a condição de solução**.
- Problema 6.2.2: *Problema da determinação de Cobertura de Vértices*
 - o **Instância**: Um grafo $G=(V,E)$ e um natural k .
 - o **Procura**: **Deseja-se V'** (subconjunto de V) tal que $|V'| \leq k$ e toda aresta de G tenha pelo menos uma extremidade em V' .

o Problema de otimização:

- Existe elemento que satisfaz a condição de solução e é um “**ótimo**”.
 - Problema 6.2.3: *Problema da Cobertura Mínima de Vértices*
 - o **Instância**: Um grafo $G=(V,E)$.
 - o **Procura**: **Deseja-se V'** (subconjunto de V) tal que toda aresta de G tenha pelo menos uma extremidade em V' e **não exista V''** (subconjunto de V) com mesma propriedade com $|V''| < |V'|$.
- Qualquer problema de localização tem um correspondente de decisão.
 - Qualquer problema de otimização tem um correspondente problema de decisão.

Problema 6.2.4: *Problema do Caixeiro Viajante (PCV)*

- o **Instância:** Um grafo $G = (\{c_1, c_2, \dots, c_n\}, E)$ e uma função d tal que, se $(c_i, c_j) \in E$, então $d(c_i, c_j) \in \mathbb{Z}^+$ é a distância entre as cidades c_i e c_j .
- o **Ótimo:** Deseja-se c , um caminho fechado em G com custo mínimo.
- Problema de otimização, cuja versão de decisão, quando todas as arestas têm o mesmo custo, é o problema do Circuito Hamiltoniano.

Problema 6.2.5: *Problema da existência de Circuito Hamiltoniano*

- o **Instância:** Um grafo $G = (V, E)$.
- o **Pergunta:** G tem circuito hamiltoniano?

Problema 6.2.6: *Problema de decisão associado ao problema do Caixeiro Viajante*

- o **Instância:** Um grafo $G = (V, E)$, com arestas valoradas e um inteiro positivo k .
- o **Pergunta:** G tem circuito hamiltoniano de custo menor ou igual a k ?
- Um algoritmo que resolve um problema de localização ou de otimização pode ser modificado para resolver um problema de decisão associado.
- As classes de problemas P , NP , NP -Completa, são definidas para problemas de decisão (por serem mais simples).
- Os requisitos de tempo de um algoritmo são expressos em termos do “tamanho da entrada” porque se espera que a dificuldade do problema varie rigorosamente com essa medida.
- **Tamanho da entrada** de uma instância I de um problema Π é definido como o número de símbolos na descrição de I obtido em um esquema de codificação do problema Π .

- Um **algoritmo** é *polinomial* se é $O(p(n))$, p um polinômio.
- Se um **algoritmo não é polinomial**, diz-se que **é exponencial**.
- Um problema de decisão é dito:
 - o **P**: resolvido **deterministicamente** em tempo polinomial.
 - o **NP**: resolvido **não deterministicamente** em tempo polinomial.

6.2.2 Algoritmos não determinísticos

- Aos comandos usuais das linguagens de programação, acrescenta-se três comandos que caracterizam o não determinismo, que são:
 - o escolhe(S): função que escolhe arbitrariamente um elemento de S.
 - o falha: sinal de terminação falha.
 - o sucesso: sinal de terminação com sucesso.
- Um algoritmo **não determinístico**:
 - o faz uso da função escolhe(S) e das instruções falha e sucesso;
 - o **termina com insucesso**, sss, não existe escolha que leve a um terminação com sucesso;
 - o tem **vários caminhos de execução**, podendo inclusive mais de um terminar em sucesso;
 - o são interessantes na solução de problemas em que **existe várias alternativas** para serem testadas.

- Exemplos de algoritmos não determinísticos:

o *Algoritmo 6.2.1: Procura*

Algoritmo de procura de um elemento x em $A(1:n)$, $n \geq 1$.

Dado x , A , procura em A o elemento x ; devolve j tal que $A(j) = x$ ou $j = 0$ se x não está em A .

Procedimento Procura(A : vetor, x : elemento)

{*Entrada*: A : vetor, x : elemento *Saída*: j ou 0 : valor}

1. $j \leftarrow$ escolhe($1:n$) {escolhe posição}
2. se $A(j) = x$ {testa se é o elemento procurado}
3. então imprime(j); sucesso {fim com sucesso}
4. senão imprime("0"); falha {fim com falha}
5. fim-se {2: $A(j) = x$?}
6. fim-Procedimento {fim Procura}

→ A complexidade do algoritmo é $O(1)$.

o Algoritmo 6.2.2: Nclassifica

Classifica os elementos do vetor $A(i)$ $1 \leq i \leq n$. A contém inteiros positivos. $Nclassifica(A, n)$ coloca os elementos de A em ordem não crescente e imprime-os nessa ordem. É usado um vetor auxiliar B .

Procedimento $Nclassifica(A: \text{vetor}, n: \text{Nat})$

{*Entrada*: vetor: $[1..n]$ (a ordenar A); *Nat*: n
Saída: vetor: B (vetor A classificado)}

1. $B \leftarrow 0$	{inicializa B com zeros}
2. <u>para</u> i <u>de</u> 1 <u>até</u> n <u>faça</u>	{preenche B : 2 a 6}
3. $j \leftarrow \text{escolhe}(1:n)$	{escolhe posição para atribuir $A(i)$ }
4. <u>se</u> $B(j) \neq 0$ <u>então</u> <u>falha</u> <u>fim-se</u>	{verifica se a posição está preenchida}
5. $B(j) \leftarrow A(i)$	{atribui $A(i)$ }
6. <u>fim-para</u>	{2: B preenchido}
7. <u>para</u> i <u>de</u> 1 <u>até</u> $n-1$ <u>faça</u>	{verifica se B está classificado}
8. <u>se</u> $B(i) > B(i+1)$ <u>então</u> <u>falha</u> <u>fim-se</u>	{falha se B não está classificado}
9. <u>fim-para</u>	{7: B está classificado}
10. <u>retorne-saída</u> (B)	{dá saída a B }
11. <u>sucesso</u>	{termina com sucesso}
12. <u>fim-Procedimento</u>	{fim $Nclassifica$ }

→ A complexidade do algoritmo é $O(n)$.

o Algoritmo 6.2.3: Cobertura de Vértices

Algoritmo para o Problema da Cobertura de Vértices (problema 6.4.2).

Dado $G = (V, E)$ e um inteiro positivo k , devolve ou $\emptyset$ ou V' , um subconjunto de V tal que $|V'| \leq k$ e toda aresta de G tem pelo menos uma extremidade em V' .

Procedimento Cobertura-de-Vértice (V, E : Conjuntos, k : valor)

{Entrada: Conjunto: V, E : k

Saída: Conjunto: V' ou $\emptyset$ }

1. $V' \leftarrow \emptyset$ {inicializa cobertura}
2. $j \leftarrow \text{escolhe}(1..k)$ {escolhe j , cardinalidade de cobertura}
3. para i de 1 até j faça {escolhe j vértices para a cobertura: de 3 a 7}
4. $v \leftarrow \text{escolhe}(V)$ {escolhe um vértice}
5. $V \leftarrow V - \{v\}$ {retira o vértice escolhido do conjunto de opções}
6. $V' \leftarrow V' \cup \{v\}$ {coloca o vértice escolhido na cobertura}
7. fim-para {3: cobertura completa}
8. para $(v, w) \in E$ faça {verifica se V' é realmente cobertura: de 8 a 10}
9. se $v \notin V'$ e $w \notin V'$ então $V' \leftarrow \emptyset$; falha fim-se { V' não cobertura, pára c/ falha}
10. fim-para {8: certificação da cobertura V' }
11. sucesso {fim com sucesso}
12. retorne saída(V') {dá saída à cobertura V' }
13. fim-Procedimento {fim Cobertura-de-Vértices}

6.2.3 Formalização do conceito de algoritmo

- Um **algoritmo** é um programa para uma Máquina de Turing Determinística (MTD) que sempre pára.

Máquina de Turing determinística (MTD)

- $MTD = (Q, \Gamma, \Sigma, \delta)$
 - o **Conjunto de estados finitos** $(Q) = \{q_o, q_s, q_n\}$
 - o **Alfabeto finito da fita** $(\Gamma) = \Sigma \subseteq \Gamma$, o alfabeto finito de entrada; $b \in \Gamma - \Sigma$, símbolo branco.
 - o Função de transição $\delta: (Q - \{q_s, q_n\}) \times \Gamma \rightarrow Q \times (\Gamma - \{b\}) \times \{1, -1\}$

MTD

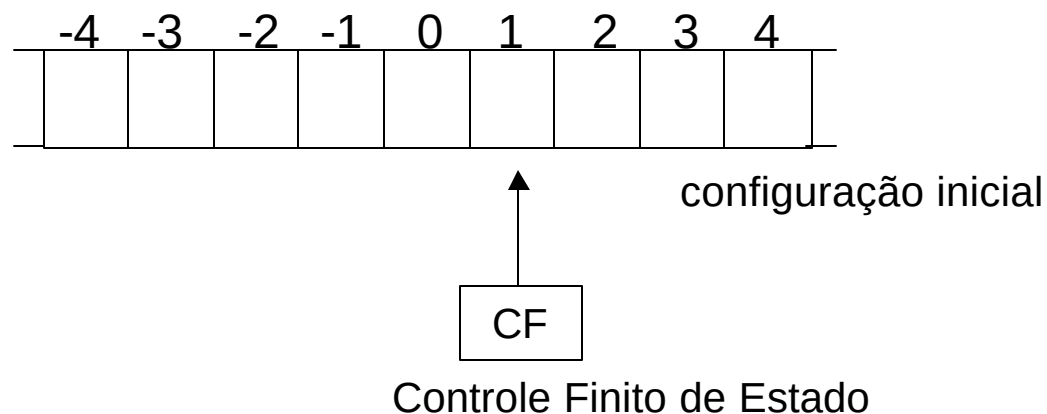


Figura 6.2.1: Máquina de Turing Determinística

- Uma MTD (ou um programa) M **aceita** $x \in \Sigma^*$ sse M pára no estado q_s , iniciando com x , e a **linguagem reconhecida** é $L_M = \{x \in \Sigma^* / M \text{ aceita } x\}$.
- O **tempo de computação** de M para x é o número de trocas de estados, até a máquina aceitar x .
- A complexidade de uma máquina M é definida pela função:

$$o \ T_M: \mathbb{N} \rightarrow \mathbb{N}$$

- o $T_M(n) = \text{Máx} \{m / \exists x \in \Sigma^*, \text{ com } |x|=n, \text{ tal que o tempo de computação de } M \text{ para } x \text{ é } m\}$
- o **M é de tempo polinomial** se existe um polinômio p tal que, para $\forall n \in \mathbb{N}$, $T_M(n)$ é $O(p(n))$.
- o **Classe P**
 $P = \{L / \text{existe um MTD } M \text{ de tempo polinomial t.q. } L = L_M\}$

Máquina de Turing não determinística (MTND)

- Um programa para uma MTND é definido exatamente como um programa para uma MTD, diferindo somente na execução.
- Uma MTND M tem dois cabeçotes, CF e MA.
- A execução da MTND:
 - o o cabeçote do MA pode escrever um símbolo de Γ e mover-se para a esquerda, ou

o ativar o cabeçote do CF, no estado inicial q_0 .

MTND

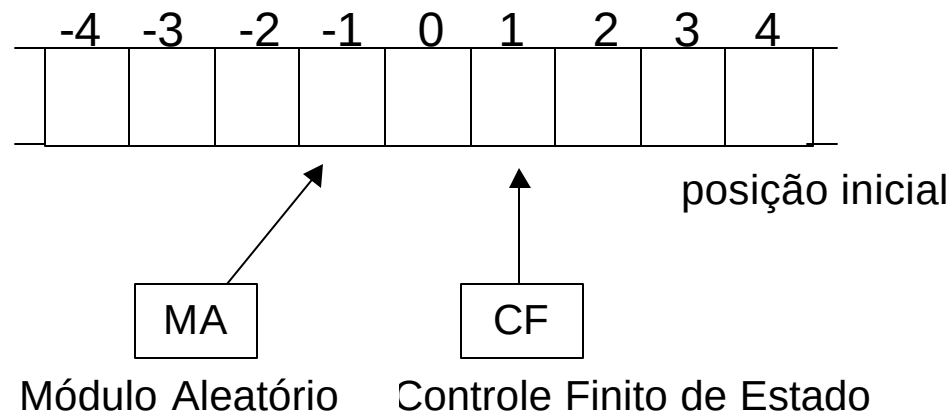


Figura 6.2.2: Máquina de Turing não determinística

- Um MTND M **aceita** x se pelo menos uma entre todas possíveis computações de M pára ao atingir um estado de aceitação, e a **linguagem reconhecida** é $L = \{ x \in \Sigma^* / M \text{ aceita } x \}$.
- O **tempo requerido pela computação** de M para a entrada x é o número mínimo de passos entre todas as computações que aceitam x .

- A complexidade de uma máquina M
 - o $T_M(n) = \text{Máx} \{ \{1\} \cup \{ m / \exists x \in L_M, \text{ com } |x| = n \text{ t.q. } m \text{ é o tempo de computação para } M \text{ aceitar } x \}$
 - o **Classe NP**
 $NP = \{ L / \exists M \text{ MTND de tempo polinomial t.q. } L_M = L \}.$
- A MTND tem a **mesma expressividade** da MTD.
- A **complexidade** das máquinas, entretanto, **pode ser diferente**, pois as MTND costumam ser mais rápidas.
- Para um mesmo problema de decisão resolvido em:
 - o **MTND** $\rightarrow$ tempo polinomial.
 - o **MTD** $\rightarrow$ tempo exponencial.
- Está em aberto a questão da existência de MTD de tempo polinomial.

Problema 6.2.7: Problema da “Satisfiability” (SAT)

- o **Instância**: U conjunto de variáveis; C coleção de cláusulas sobre U.
- o **Pergunta**: Existe uma atribuição satisfaz C?
- A **certificação** de um problema de decisão é o teste de uma candidata a solução I é a verificação da candidata como solução do problema proposto.
- A Certificação é certamente um processo bem mais rápido que a solução do problema, que pode requerer um número não polinomial de certificações.
- **P** classe de problemas que podem ser **resolvidos eficientemente**.
- **NP** classe de problemas que podem ser certificados **eficientemente**.
- Os problemas de NP possuem algoritmos determinísticos de ordem exponencial, como é mostrado no teorema 6.2.1.

Teorema 6.2.1: Se $\Pi \in \text{NP}$, então existe um polinômio p tal que Π pode ser resolvido por algoritmo determinístico de complexidade $O(2^{p(n)})$.

6.2.4 Relação entre P e NP

- A relação entre P e NP é um **problema em aberto** na Ciência da Computação.
- Certamente $P \subseteq NP$, mas não se sabe se $P = NP$. Existe uma forte conjectura de que $P \neq NP$.
- Dado $\Pi \in NP$, se $NP - P \neq \emptyset$ há duas possibilidades para Π :
 - o $\Pi \in P$
 - o $\Pi \in NP - P$
- Mas $\Pi \in NP - P$ implica que $NP - P \neq \emptyset$. Considere a afirmação conversas:

Se $NP - P \neq \emptyset$ então existe Π , $\Pi \in NP - P$.

- Essa afirmação motivou a criação da classe dos problemas **mais difíceis de NP**, a classe dos problemas **NP-Completo**.

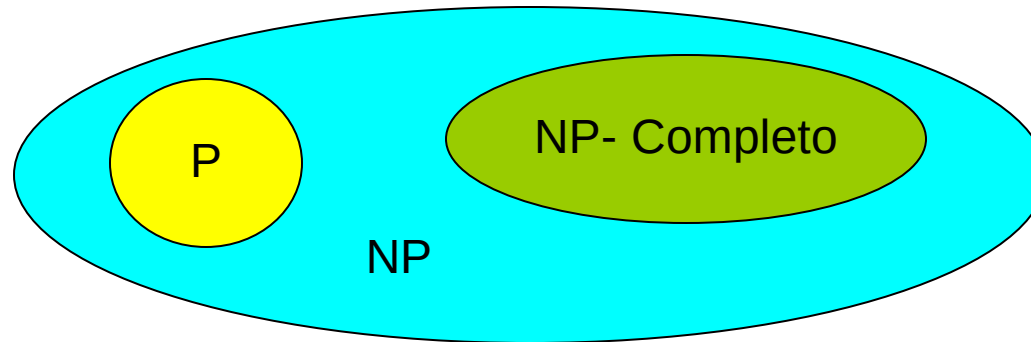


Figura 6.2.3: P NP

NP-Compleitude

- Se um problema NP-Completo tiver solução determinística polinomial, qualquer outro problema NP também terá.

- Redução polinomial ($\propto$):

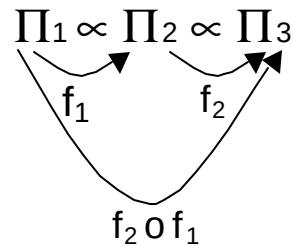
o de uma linguagem $L_1 \subseteq \Sigma_1^*$ para uma linguagem $L_2 \subseteq \Sigma_2^*$ é uma função $f : \Sigma_1^* \rightarrow \Sigma_2^*$ que satisfaz as duas condições seguintes:

(i) f é polinomial.

(ii) para todo $x \in \Sigma_1^*$, $x \in L_1$ sss $f(x) \in L_2$.

- Se existe uma redução polinomial de L_1 para L_2 , escreve-se $L_1 \propto L_2$, e diz-se que L_1 **é redutível a** L_2 .
- Se existe uma redução polinomial de um problema Π_1 para um problema Π_2 , escreve-se $\Pi_1 \propto \Pi_2$ e diz-se que Π_1 **é redutível a** Π_2 .

Lema 6.2.1: ∞ é transitiva.



Lema 6.2.2: Seja os problemas Π_1, Π_2 tal que se $\Pi_1 \infty \Pi_2$ então

- (i) Se $\Pi_2 \in P$ então $\Pi_1 \in P$.
- (ii) Se $\Pi_2 \in NP$ então $\Pi_1 \in NP$.
- Sejam Π_1 e Π_2 dois problemas. Π_1 e Π_2 são **polinomialmente equivalentes** sss $\Pi_1 \infty \Pi_2$ e $\Pi_2 \infty \Pi_1$.
- Um problema Π é dito NP-Completo se $\Pi \in NP$ e, para qualquer outro $\Pi' \in NP$, $\Pi' \infty \Pi$.
- Os problemas NP-Completos são os **mais difíceis** entre os problemas NP.

Lema 6.2.3: Se um o problema Π é NP-completo, então $\Pi \in P$ sss $P = NP$.

- O grande número de problemas NP-Completo e o fato de ninguém ter conseguido criar um algoritmo polinomial para um deles torna mais forte a conjectura de que $P \neq NP$.
- Outras questões referentes a essas classes de problemas também estão em aberto, por exemplo:

o A classe NP é fechada para o complemento? Isto é, se a linguagem L sobre o alfabeto Σ é NP, será que $L' = \Sigma^* - L$ é NP?

Classe Co-NP

- Classe de todas as linguagens L tais que $L' \in NP$.
- A questão é reescrita como: **NP = Co-NP?**
- Como P é fechada sob complemento, então $P \subseteq NP \cap \text{Co-NP}$. E a questão fica: $P = NP \cap \text{Co-NP}$? ou $(NP \cap \text{Co-NP}) - P \neq \emptyset$?

- Existem quatro relações possíveis entre essas classes:

o $P = NP$ e, portanto, $NP = \text{Co-NP}$

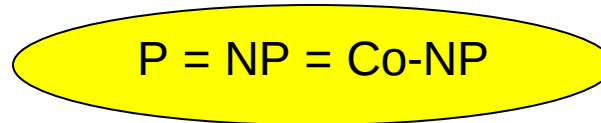


Figura 6.2.4: $P = NP$

o Se $P \neq NP$ então há três possibilidades:

- $NP = \text{Co-NP}$, mas $P \neq NP$



Figura 6.2.5: $P \neq NP = \text{Co-NP}$

- $NP \neq Co-NP$ e $P = NP \cap Co-NP$

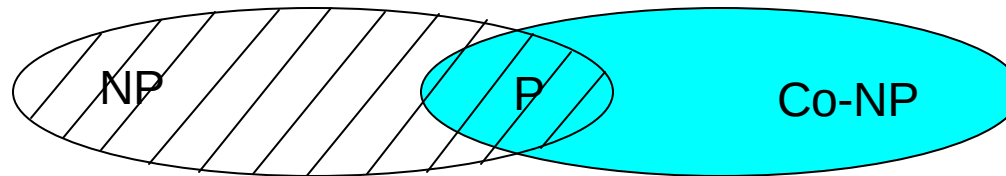


Figura 6.2.6: $NP \cap Co-NP = P \neq NP \neq Co-NP$

- $NP \neq Co-NP$ e $P \neq NP \cap Co-NP$

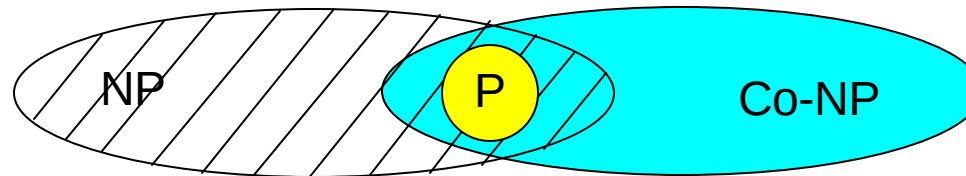


Figura 6.2.7: $P \neq NP \neq Co-NP \neq NP \cap Co-NP$

→ Problema: “x é primo?”.

→ Problema de Programação Linear

- Se $\Pi \in \text{NP-Completo}$, então não se conhece solução eficiente para o problema.

Teorema de Cook

- Os teoremas a seguir apresentam **uma maneira de identificar** um problema NP-Completo.

Teorema 6.2.2: Se Π_1 é NP-Completo, $\Pi_2 \in \text{NP}$ e $\Pi_1 \propto \Pi_2$, então Π_2 é NP-Completo.

- *Demonstração:* Sejam $\Pi_1, \Pi_2 \in \text{NP}$ tal que: Π_1 é NP-Completo e $\Pi_1 \propto \Pi_2$. Seja $\Pi \in \text{NP}$, como Π_1 é NP-Completo, $\Pi \propto \Pi_1$. Mas $\Pi_1 \propto \Pi_2$, e como $\propto$ é transitivo, $\Pi \propto \Pi_2$, logo Π_2 é NP-Completo.

Teorema 6.2.3 - Teorema de Cook: O problema da "Satisfiability" (SAT) é NP-Completo.

- *Demonstração:* É preciso mostrar que:

(i) SAT é NP (trivial)

(ii) $\forall \Pi \in \text{NP}, \Pi \leq \text{SAT}$

o Para mostrar (ii), considera-se a linguagem L_{SAT} e mostra-se que para $\forall L \in \text{NP}, L \leq L_{\text{SAT}}$.

o $L \in \text{NP}$ significa que **L é reconhecida por uma MTND** $M=(Q, \Gamma, \Sigma, \delta)$, tal que $L(M) = L$.

o Sejam:

- $Q = \{q_0, q_1, q_2, q_3, \dots, q_r\}$, com $r = |Q| - 1$; $q_1 = q_S$ e $q_2 = q_N$
- $\Gamma = \{s_0, s_1, s_1, \dots, s_v\}$, onde $v = |\Gamma| - 1$; $s_0 = b$

o função $f_L = \Sigma^* \rightarrow \Sigma_{SAT}^*$, com a seguinte propriedade:

- $\forall x \in \Sigma^*, x \in L \text{ sss } f_L(x) \text{ tem uma atribuição verdadeira que a satisfaz.}$
- o Se $x \in \Sigma^*$ é aceito por M , então existe uma computação de M que aceita x em, no máximo, $p(n)$ passos, onde $n = |x|$ e p é um polinômio.
- o A partir da computação de x , será construído $f_L(x)$: um conjunto de variáveis e de cláusulas, uma instância de SAT com resposta SIM.
- o $f_L(x)$ conterá um conjunto de cláusulas que simularão a computação de x em M .
- o A construção de f_L garantirá que o conjunto de cláusulas de $f_L(x)$ terá uma atribuição verdadeira sss M aceitar x .

o Variáveis do conjunto U:

- $Q[i, k]$: com $0 \leq i \leq p(n)$ e $0 \leq k \leq r$, significando que, ao completar o i° passo, M estará no estado q_k .
- $H[i, j]$: com $0 \leq i \leq p(n)$ e $-p(n) \leq j \leq p(n)+1$, significando que, ao completar o i° passo, o cabeçote CF estará apontando para a posição j da fita.
- $S[i, j, k]$: com $0 \leq i \leq p(n)$, $-p(n) \leq j \leq p(n)+1$ e $0 \leq k \leq v$, significando que ao completar o i° passo o conteúdo da posição j da fita será o símbolo s_k .
- $|U| = (p(n)+1)(r+1) + (p(n)+1)(2p(n)+2) + (p(n)+1)(2p(n)+2)(v+1) = O(p(n))^2$.

o Conjunto C de cláusulas de $f_L(x)$

Grupo 1:

→ $\{Q[i, 0], Q[i, 1], \dots, Q[i, r]\}$, $0 \leq i \leq p(n)$
garantem que **M estará em um estado**, ao completar o i^o passo.

→ $\{\neg Q[i, j], \neg Q[i, j']\}$, $0 \leq i \leq p(n)$ e $0 \leq j \leq j' \leq r$
garantem que **M não estará em mais de um estado** ao completar o i^o passo.

Grupo 2:

→ $\{H[i, -p(n)], H[i, -p(n)+1], \dots, H[i, p(n)+1]\}$, $0 \leq i \leq p(n)$

→ $\{\neg H[i, j], \neg H[i, j']\}$, $0 \leq i \leq p(n)$, $-p(n) \leq j \leq j' \leq v$

Ao completar o i^o passo, o **cabeçote CF estará apontando para exatamente uma posição**.

Grupo 3:

$\rightarrow \{S[i, j, 0], S[i, j, 1], \dots, S[i, j, v]\}, 0 \leq i \leq p(n) \text{ e } -p(n) \leq j \leq p(n)+1$

$\rightarrow \{\neg S[i, j, k], \neg S[i, j, k']\}, 0 \leq i \leq p(n), -p(n) \leq j \leq p(n)+1 \text{ e } 0 \leq k \leq k' \leq v$

Ao completar o i^o passo, **cada posição da fita conterá exatamente um símbolo** de Γ .

Grupo 4:

$\rightarrow \{Q[0, 0]\}, \{H[0, 1]\}, \{S[0, 0, 0]\}$

$\rightarrow \{S[0, 1, k]\}, \{S[0, 2, k]\}, \dots, \{S[0, n, k]\}$

$\rightarrow \{S[0, n+1, 0]\}, \{S[0, n+2, 0]\}, \dots, \{S[0, p(n), 0]\}$, onde $x = s_{k1} s_{k2} \dots s_{kn}$

- Antes do primeiro passo, a computação estará na configuração inicial: M estará no estado q_0 ; o cabeçote apontando para a posição 1; na posição zero deverá ter branco; nas posições de 1 a n estará a cadeia x e da posição n+1 para a direita estará branco.

Grupo 5:

$\rightarrow \{Q[p(n), 1] \}$

No passo $p(n)$, M entrará no estado q_s e aceitará x .

Grupo 6:

Para cada quádrupla (i, j, k, e) , $0 \leq i \leq p(n)$, $-p(n) \leq j \leq p(n)+1$, $0 \leq k \leq r$ e $0 \leq e \leq v$ esse grupo contém as seguintes 3 cláusulas:

$$\rightarrow \{\neg H[i, j], \neg Q[i, k], \neg S[i, j, e], H[i+1, j+\Delta]\}$$

$$\rightarrow \{\neg H[i, j], \neg Q[i, k], \neg S[i, j, e], Q[i+1, k']\}$$

$$\rightarrow \{\neg H[i, j], \neg Q[i, k], \neg S[i, j, e], S[i+1, j, e']\}$$

com Δ , k' , e' assim definidos:

- Se $q_k \in Q - \{q_s, q_N\}$ então $(q_{k'}, s_{e'}, \Delta) = \delta(q_k, s_e)$,

- Se $q_k \in \{q_s, q_N\}$ então $\Delta = 0$, $k' = k$, $e' = e$.

A cada passo i , $0 \leq i \leq p(n)$, a configuração de M para o passo $i+1$ seguirá da aplicação da função de transição δ à configuração do passo i . Assim, fica definido o conjunto C de cláusulas geradas pela entrada x , portanto está definida a função f_L .

o $|f_L(x)| = |U| + |C|$, é $O((p(n))^4)$

$|U|$ é $O((p(n))^2)$

$|C|$ é $O((p(n))^2)$

o Conclui-se que, para toda $L \in NP$, é possível construir uma redução polinomial f_L de L em L_{SAT} , tal que para $x \in L$, sss $f_L(x)$ é verdadeira.

o A partir do SAT, mostra-se, por exemplo, que o 3SAT é NP-Completo.

Problema 6.2.8: *Problema 3SAT*

o **Instância:** Um conjunto de variáveis C ; uma coleção de cláusulas sobre U , todas com exatamente três literais.

o **Pergunta:** Existe uma atribuição verdadeira para C ?

- A partir do 3SAT, mostra-se, por exemplo, que o 3DM é NP-Completo.

Problema 6.2.9: *Problema 3DM*

- o **Instância:** Conjuntos X, Y, Z , com cardinalidade q e $M \subseteq X \times Y \times Z$.
- o **Pergunta:** Existe $M' \subseteq M$ tal que cada elemento de X, Y, Z apareça exatamente uma vez em M' ?
- A partir do 3DM, mostra-se, por exemplo, que o problema da partição é NP-Completo.

Problema 6.2.10: *Problema Partição*

- o **Instância:** Um conjunto A , $s:A \rightarrow \mathbb{Z}^*$, uma função que associa a cada $a \in A$ um inteiro positivo $s(a)$.
- o **Pergunta:** Existe $A' \subseteq A$ tal que $\sum_{a \in A'} s(a) = \sum_{a \in A - A'} s(a)$.
- Cadeia de redução polinomial: **SAT $\propto$ 3SAT $\propto$ 3DM $\propto$ Partição**

Problemas de Verificação da Comunicação Síncrona de Processos (VCSP)

- Considere um sistema concorrente com comunicação síncrona, por caixa postal, no qual as caixas postais são utilizadas somente para fins de comunicação, isto é, não contêm mensagens.
- **Instruções:** S (envio de sinais) e R (recebimento de sinais).

Problema 6.2.11: *Problemas de Verificação da Comunicação Síncrona de Processos (VCSP)*

- o **Instância:** St, um sistema de processos concorrentes com comunicação síncrona através de caixa postal; X, caixa postal.
- o **Pergunta:** É possível ocorrer, em St, sincronização pela caixa X?
- Mostrar-se que o problema **VCSP é um problema NP-Completo**, através do teorema 6.2.2.
 - (i) VCSP é NP
 - (ii) SAT $\propto$ VCSP

- *Demonstração:*

- o A prova de (i) é trivial.

- o Para provar (ii) considere U e C como sendo uma instância de SAT.

- $U = \{u_1, u_2, \dots, u_n\}$ um conjunto de variáveis.

- $C = \{c_1, c_2, \dots, c_m\}$, um conjunto de cláusulas.

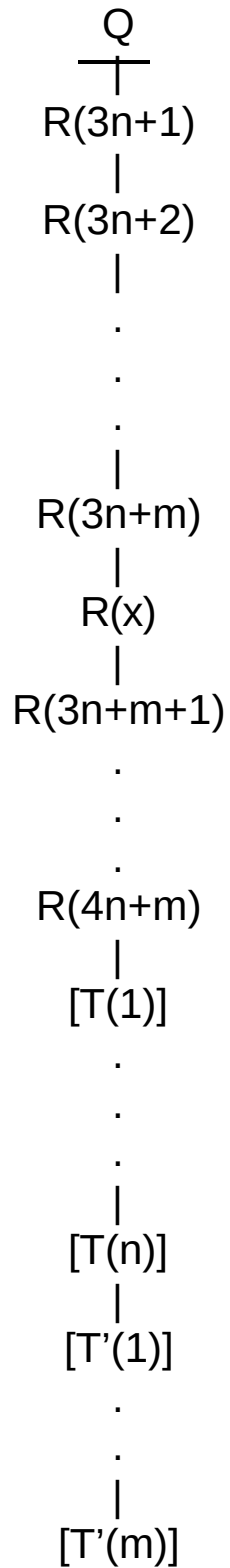
- o É preciso definir uma redução polinomial que, em tempo polinomial, descreva a partir de U e C uma instância St, X para VCSP tal que, se U tem uma interpretação que faz C verdadeira, então St tem uma possível sincronização pela caixa postal X e vice versa.

- o Os processos P e Q se comunicam pela caixa postal X.

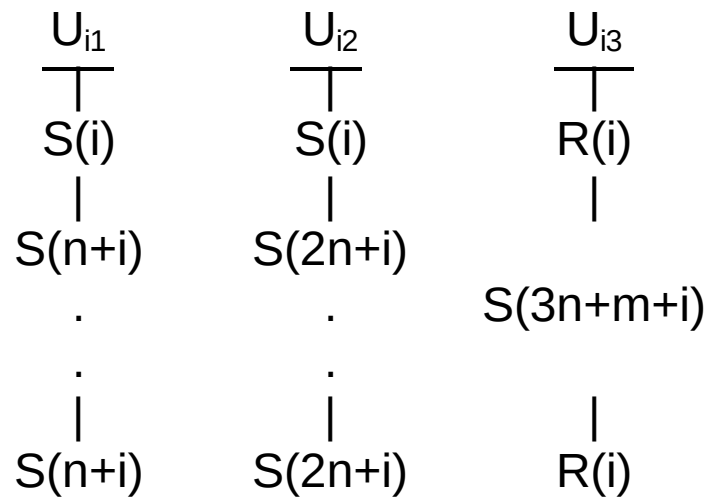
- o St será constituído por processos tipo U, tipo C, processo P e processo Q.

- o Para cada variável u_i de U , serão definidos 3 processos:
- U_{1i} , vai representar a atribuição “verdade” a u_i ;
 - U_{2i} , vai representar a atribuição “falso” para u_i ;
 - U_{3i} , vai representar a oportunidade de u_i ser verdadeira ou falsa.

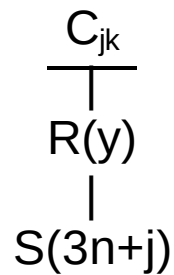
Processos Q



Processos tipo U



Processo tipo C



com $y = n+i$ se $a_{kj} = u_i$
 $y = 2n+i$ se $a_{kj} = \neg u_i$
 $j = 1, \dots, m$
 $k = 1, \dots, b_j$

Processo P

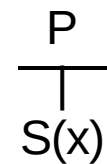


Figura 6.2.8: Problema VCSP

- o Conforme a sincronização na caixa postal i , será enviado sinal para a caixa postal $n+i$ ($u_i = \text{verdade}$) ou caixa $2n+i$ ($u_i = \text{falso}$).
- o O processo U_1 será constituído de $S(i)$ seguido de tantas ocorrências de $S(n+i)$ quantas forem as ocorrências de u_i nas cláusulas de C , ou uma ocorrência de $S(n+i)$ se u_i não ocorre nas cláusulas de C .
- o O processo U_{i2} será constituído de forma análoga: $S(i)$, seguido de tantas ocorrências de $S(2n+1)$ quantas forem as ocorrências de $\neg u_i$ nas cláusulas de C , ou uma ocorrência de $S(2n+i)$ se $\neg u_i$ não ocorre em C .
- o Para cada $c_j \in C$, $c_j = a_{1j}, a_{2j}, \dots, a_{bjj}$ são definidos b_j processos tipo C , cada um representando a possibilidade de o literal correspondente ser verdadeiro.
- o O processo Q inicia por m instruções R : $R(3n+1), R(3n+2), \dots, R(3n+m)$, onde $R(3n+j)$ representa a cláusula c_j verdadeira, depois vem $R(X)$. Assim, a execução do processo Q alcançará a instrução $R(X)$ se e somente se existir uma possibilidade de todas as cláusulas serem verdadeiras.

- o Após a instrução $R(X)$, as instruções R e S têm o objetivo de terminar a execução do sistema St . As instruções $R(3n+m+1)$, $\dots$, $R(4n+m)$ têm o objetivo de liberar a outra possibilidade de valor verdade de cada variável, cada instrução $R(3n+m+i)$ corresponde a uma instrução $S(3n+m+i)$ no processo U_{3i} .
- o $[T(i)]$ e $[T'(j)]$ são seqüências de instruções R e S , tais que:
 - $[T(i)]$ deve liberar todas as pendências com relação à variável u_i em processos tipo U .
 - haverá uma ocorrência de $R(n+i)$ se u_i não ocorrer em cláusulas de C ;
 - haverá uma ocorrência de $R(2n+i)$ se $\neg u_i$ não ocorrer em cláusulas de C .
 - $[T'(j)]$ deve liberar todas as pendências com relação a cláusulas c_j , em processos tipo C . É constituído de $b_j - 1$ ocorrências de $R(3n+j)$.

o Fazem um total de:

- Processos: $2 + 3n + \sum_{j=1}^m b_j \leq 2 + 3n + 2nm$

- Caixas postais: $4n + m$

- Instruções: $2 (5n + 2 + \sum_{j=1}^m b_j)$

o Existe uma atribuição verdade para as variáveis de U que tornam todas as cláusulas de C verdadeiras se e somente se o sistema correspondente St tiver possibilidade de sincronização na caixa X .

o Pelo número de processos, caixas postais e instruções de St , tem-se que a construção de St é possível em tempo polinomial.

Os exemplos 6.2.1 e 6.2.2 ilustram o procedimento.

Exemplo 6.2.1

$\frac{P}{S(x)}$

$\frac{Q}{R(10)}$
 $|$
 $R(11)$
 $|$
 $R(X)$
 $|$
 $R(12)$
 $|$
 $R(13)$
 $|$
 $R(14)$
 $|$
 $R(5)$
 $|$
 $R(9)$
 $|$
 $R(10)$
 $|$
 $R(10)$
 $|$
 $R(11)$

$\frac{U_{11}}{S(1)}$
 $|$
 $S(4)$

$\frac{U_{21}}{S(1)}$
 $|$
 $S(7)$

$\frac{U_{31}}{R(1)}$
 $|$
 $S(12)$
 $|$
 $R(1)$

$\frac{U_{12}}{S(2)}$
 $|$
 $S(5)$

$\frac{U_{22}}{S(2)}$
 $|$
 $S(8)$
 $|$
 $S(8)$

$\frac{U_{32}}{R(2)}$
 $|$
 $S(13)$
 $|$
 $R(2)$

$\frac{U_{13}}{S(3)}$
 $|$
 $S(6)$

$\frac{U_{23}}{S(3)}$
 $|$
 $S(9)$

$\frac{U_{33}}{R(3)}$
 $|$
 $S(14)$
 $|$
 $R(3)$

$\frac{C_{11}}{R(4)}$
 $|$
 $S(10)$

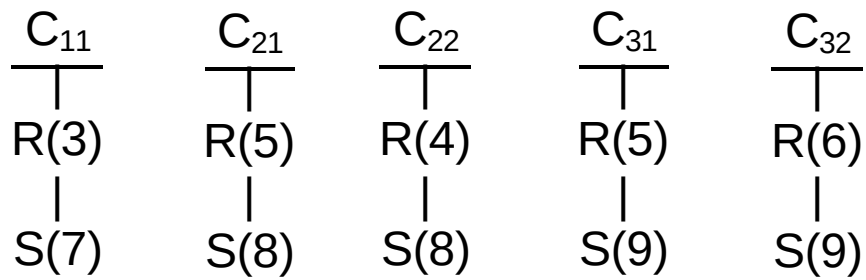
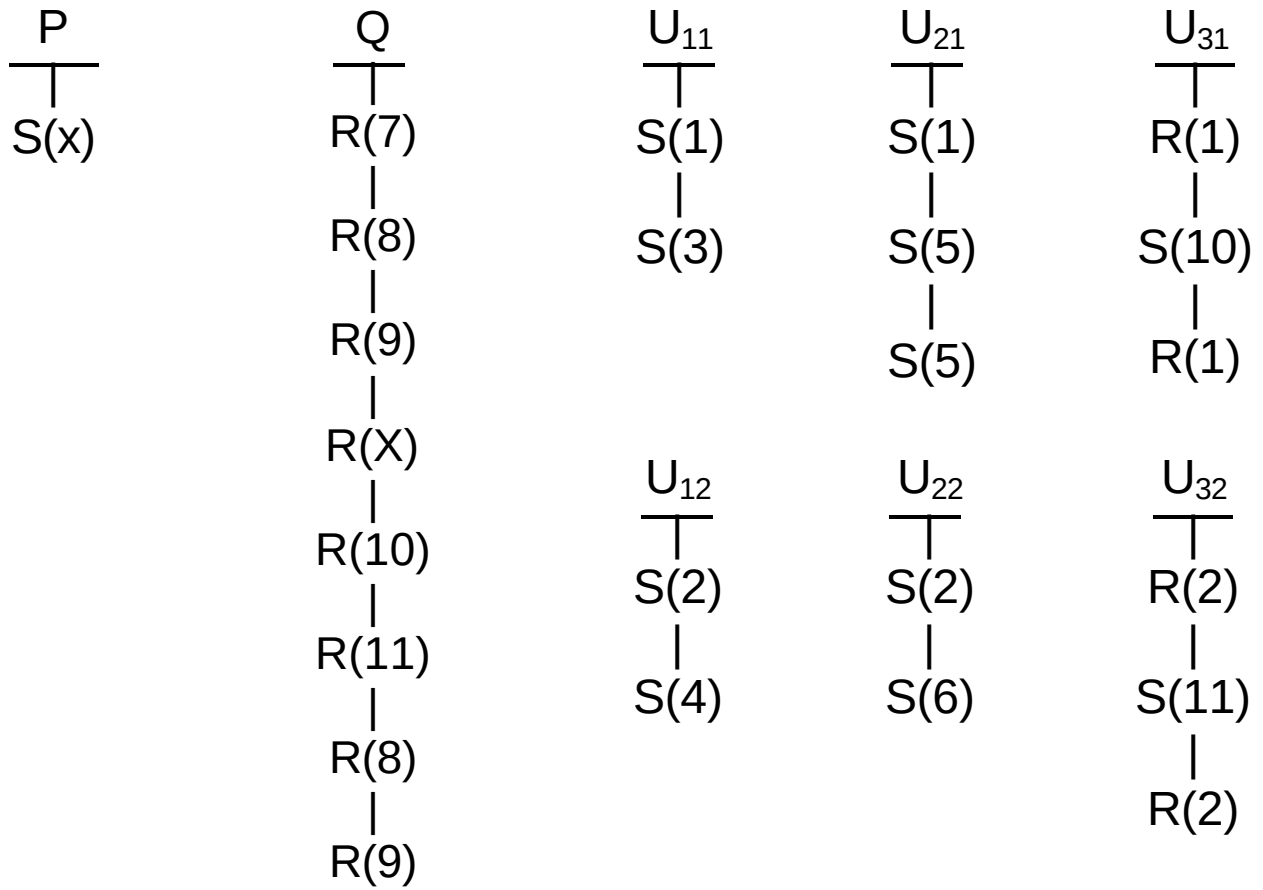
$\frac{C_{12}}{R(8)}$
 $|$
 $S(10)$

$\frac{C_{13}}{R(6)}$
 $|$
 $S(10)$

$\frac{C_{21}}{R(7)}$
 $|$
 $S(11)$

$\frac{C_{22}}{R(8)}$
 $|$
 $S(11)$

Exemplo 6.2.2



6.2.5 Algoritmos Pseudopolinomiais

- Um algoritmo A é dito de **complexidade pseudopolinomial** se sua complexidade é polinomial para o tamanho da entrada, quando essa é codificada em **unário**.
- Um algoritmo A é dito de **complexidade pseudopolinomial** quando a complexidade é polinomial no **valor da entrada** (e não no tamanho da entrada).
- **NP-Completo-fraco** - Problema NP-Completo que **admite algoritmo pseudopolinomial**.
- **NP-Completo-fortes** – Problema NP-Completo que não admitem algoritmo pseudopolinomial (exceto se $P \neq NP$).
 - o Caso exista um algoritmo pseudopolinomial, para eles, implica que $P=NP$. Ex.: *Problema do Circuito Hamiltoniano*.

6.2.6 Outras Classes de Problemas

- Existem problemas que estão numa classe “meio nebulosa”, como os de:
 - o decidir se duas expressões regulares são equivalentes;
 - o decidir se uma palavra é gerada por uma certa gramática sensível ao contexto.
- **Problemas NP-difíceis:**
 - o são problemas polinomialmente reduzíveis de problemas NP-Completos, entretanto não se conhece algoritmo não determinístico de tempo polinomial que os resolva.
 - o Em geral não se sabe se pertencem ou não a NP.
 - o se pertence a NP é NP-Completo.
 - o Um problema Π é **NP-Difícil** se, para qualquer $\Pi' \in \text{NP}$, $\Pi' \leq \Pi$.

- Os problemas NP-Difíceis têm a propriedade de **não serem resolvíveis deterministicamente em tempo polinomial exceto se $P=NP$** , mas não se sabe se são NP ou não.
- São potencialmente mais difíceis que os NP-Completos.

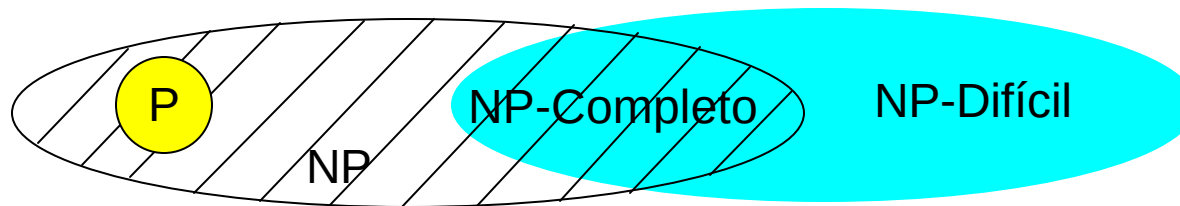


Figura 6.2.11: $NP\text{-}Completo = NP \cap NP\text{-}Difícil$

- Se $P \neq NP$, será que $NP\text{-}P = NP\text{-}Completo$?
- Existem problemas que são problemas NP, mas não foi possível até o momento reduzi-los polinomialmente de um problema NP-Completo, nem se conhecem algoritmos polinomiais que os resolvam. Exemplos:
 - o decidir se dois grafos são isomorfos;
 - o saber se um inteiro é primo.

- Esses problemas são fortes candidatos a estar em NP entre P e NP-Completo.
- **Classe NDI** - classe intermediária em NP, que é formada por problemas **mais difíceis que os de P**, mas **menos difíceis que os de NP-Completo**.

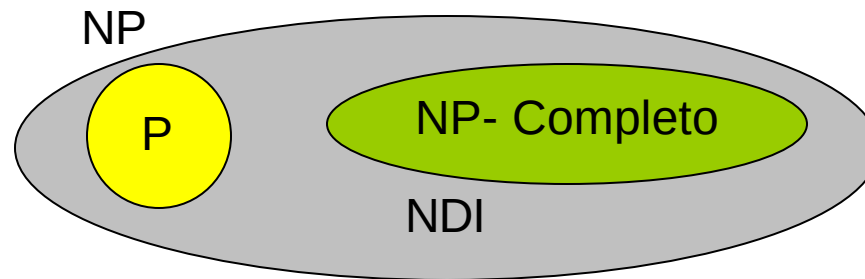


Figura 6.2.12: $NDI = NP - (P \cup NP\text{-Completo})$

- Se $P \neq NP$, então $NDI \neq \emptyset$ porém, não se conhece nenhum problema real em NDI.
- Há ainda problemas que não se sabe se são NP-Difíceis, como o de decidir se uma expressão lógica é ou não uma tautologia.

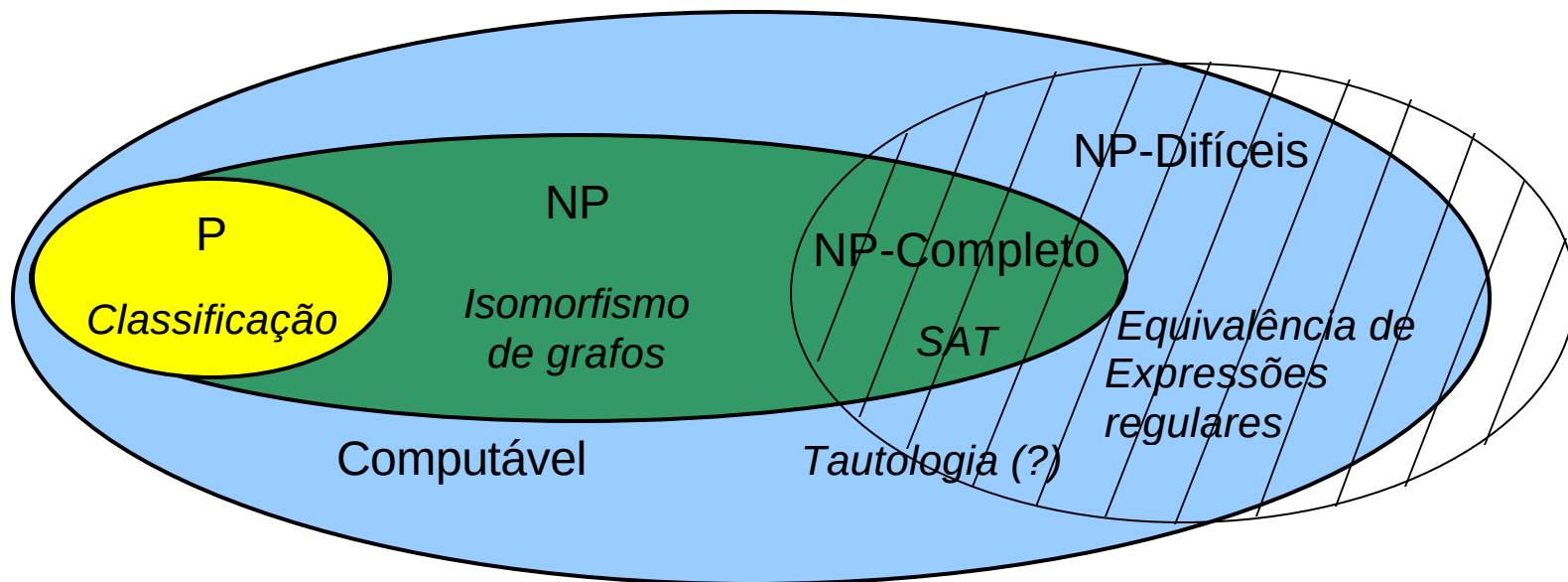


Figura 6.2.13: Exemplos em classes de problemas

Complexidade de Espaço (memória)

- **Classe Pspace**

o classe de problemas resolvidos num espaço polinomial, que requerem um número polinomial em n (tamanho da entrada) de unidades de memória.

o sua relação com NP e Co-NP é ilustrada na figura 6.2.14.

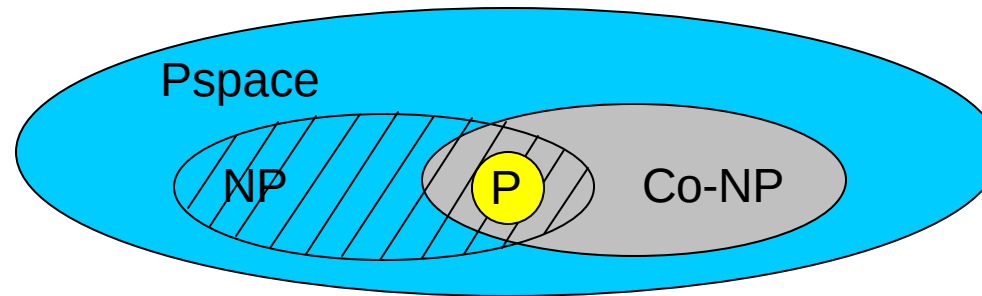


Figura 6.2.14: $\text{Pspace} \supset \text{NP} \cup \text{Co-NP}$

- Questão em aberto: “existe problema resolvido em espaço polinomial que não pode ser resolvido em tempo polinomial?”.
- $\text{Pspace} - (\text{NP} \cup \text{Co-NP}) \neq \emptyset$?
- Definir Co-NP

Relações de inclusão entre as principais classes de problemas

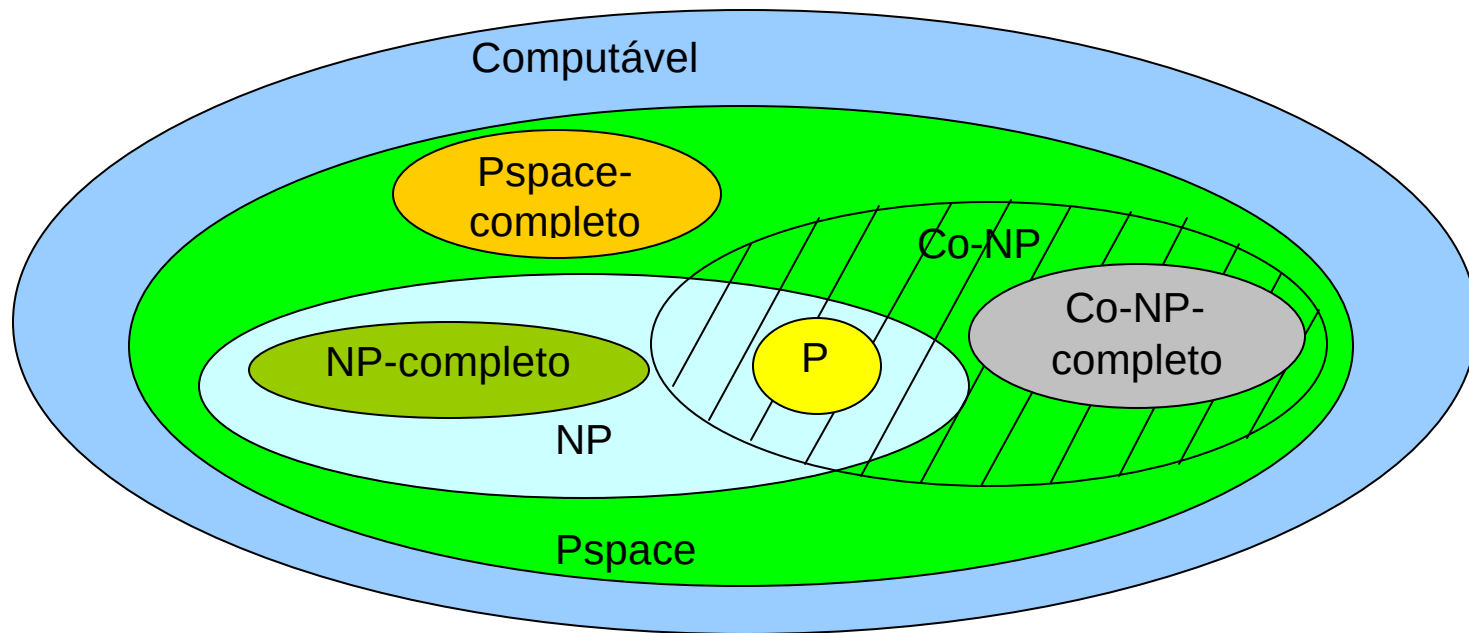


Figura 6.2.15: Hierarquia de Classes de Problemas

- A noção de completude aplica-se a qualquer uma dessas classes de problemas.
- A classe dos **problemas computáveis** é a classe dos problemas para os quais existe algoritmo que o resolva.

6.3 RESUMO DO CAPÍTULO

- O objetivo deste capítulo é estudar a **complexidade inerente ao problema**.
- **Limite inferior** - é um **resultado teórico** que determina não ser possível desenvolver um algoritmo para o problema com complexidade melhor que uma certa função.
- **Limite superior** - refere-se ao **melhor algoritmo conhecido** que resolve um determinado problema.
- Se o **limite inferior é igual ao limite superior**, diz-se que o **problema está fechado** (quanto à complexidade), e a essa função chama-se **complexidade mínima do problema**. Ex.: **Problema de Classificação**, tanto no pior caso como no caso médio, com complexidade mínima $\Omega(n \log n)$.
- **Problema tratável** - tem um **algoritmo polinomial** que o resolve.
- **Problema intratável** - só tem **algoritmos de complexidade exponencial** que o resolve.

- **Problemas de decisão** – se têm **resposta SIM/NÃO**.
- **Problemas de localização** - envolvem a **procura de um elemento que satisfaça a condição** de solução.
- **Problemas de otimização** - requerem uma **qualidade ótima da solução**.
- As mais **importantes classes de problemas**, P, NP e NP-Completa, são definidas para problemas de decisão.
- Algoritmos que resolvem um problema de localização ou de otimização podem ser modificados para resolver problemas de decisão associados. Permitindo que questões como **intratabilidade sejam estudadas no âmbito dos problemas de decisão**.
- Um problema de decisão é dito:
 - o **P** se é resolvível **deterministicamente em tempo polinomial** e
 - o **NP** se é resolvível **não deterministicamente em tempo polinomial**.

- Um algoritmo que faz uso da função escolhe(S) e das instruções falha e sucesso é **não determinístico**.
- A noção de algoritmo é formalizada através do modelo de computação da Máquina de Turing.
 - o Um algoritmo é um programa para um MTD que sempre pára.
 - o Um algoritmo não determinístico é um programa para um MTND que sempre pára.

Teorema 6.2.1: Se $\Pi \in \text{NP}$, então existe um polinômio p tal que Π pode ser resolvido por algoritmo determinístico de complexidade $O(2^{p(n)})$.

- Um problema Π é dito NP-Completo se $\Pi \in \text{NP}$ e, para qualquer outro $\Pi' \in \text{NP}$, $\Pi' \leq \Pi$.
- Os problemas NP-Completos são os **mais difíceis** entre os problemas de NP.

Lema 6.2.3: Se um o problema Π é NP-completo, então $\Pi \in P$ sss $P = NP$.

- A **classe Co-NP** é a classe de todas as linguagens L tais que $L' \in NP$ (L' é o complemento de L).
- Muitas questões envolvendo **relações entre classes** de problemas estão **em aberto**, como:
 - o $P = NP$?
 - o $NP = Co-NP$?
 - o $P = NP \cap Co-NP$?
- O **Teorema de Cook** estabeleceu: “O problema da **"Satisfiability" (SAT)** é **NP-Completo**”.

Teorema 6.2.2: Se Π_1 é NP-Completo, $\Pi_2 \in NP$ e $\Pi_1 \leq \Pi_2$, então Π_2 é NP-Completo.

- Um algoritmo A é dito de **complexidade pseudopolinomial** se sua complexidade é polinomial para o tamanho da entrada, quando esta é codificada em unário. Isto é, a complexidade é polinomial no valor da entrada (e não no tamanho da entrada).
- **NP-Completo-fraco** - problema NP-Completo que admite algoritmo pseudopolinomial.
- **NP-Completo-forte** - problema cuja existência de algoritmo pseudopolinomial implica que $P=NP$.
- Um problema Π é **NP-Difícil** se, para qualquer $\Pi' \in NP$, $\Pi' \leq \Pi$.
- Os problemas NP-Difíceis têm a propriedade de **não serem resolvíveis deterministicamente em tempo polinomial exceto se $P=NP$** , mas não se sabe se são NP ou não. Esses problemas são potencialmente mais difíceis que os NP-Completos.